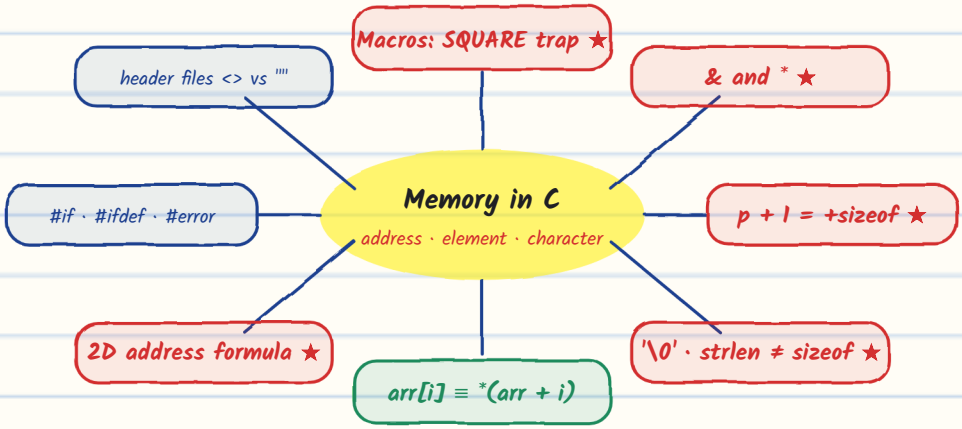
Addresses, Arrays & Strings in C

Header files · Preprocessor · Macros · Pointers · 1D/2D arrays · Strings: exam-ready in 11 sheets



↑ the whole lesson on one page: red = high-yield, green = the rule everything else hangs on

➜ Syllabus coverage

The Structured Programming Language syllabus lists: "Header files; Preprocessor; Pointers and arrays; Strings; Multidimensional array". Data types, operators and control flow, and functions, scope and storage classes, were the previous lessons; structures, unions and enums, then I/O, files and `malloc`, come next.

High-yield list

① <> system dirs; "" own dir first
② Preprocessor = text work before compiling
③ SQUARE(2+3) with x*x → 11
④ & address-of; * value-at
⑤ p + 1 moves sizeof(*p) bytes
⑥ arr[i] ≡ *(arr+i) ≡ i[arr]
⑦ sizeof(arr) ≠ sizeof(p)
⑧ C is row-major
⑨ Strings end in '\0' (ASCII 0)
⑩ gets() removed in C11 → fgets()

Legend yellow = key term · red circle = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

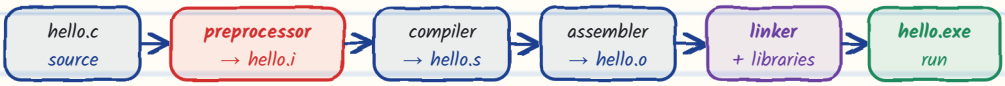
➜ Contents

✓ Header files & the build pipeline … p. 2
✓ Preprocessor directives & macros … p. 3
✓ Pointers: &, *, kinds, const … p. 4
✓ Pointer arithmetic … p. 5
✓ 1D arrays & the array-pointer link … p. 6
✓ 2D arrays & address formula … p. 7
✓ Strings and '\0' … p. 8
✓ string.h & string input … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

1 Header Files and the Build Pipeline

MCQ favourite ★

Definition A **header file** (`.h`) holds **declarations** shared by source files: function prototypes, macros, constants and type definitions (`typedef`, `struct`). `#include` pastes the file's whole text in place of the line **before compilation**. A header holds no compiled code: the bodies of `printf`, `sqrt` … live in the **library**, joined in by the **linker**.



headers are pasted in at the preprocessor; library code joins at the linker

gcc -E stops after preprocessing, -S after compiling, -c after assembling

Header	What it gives you · names to recognise
`<stdio.h>`	standard I/O: printf, scanf, puts, fgets, getchar, fopen, FILE, EOF
`<conio.h>`	console I/O, **not standard C** (Turbo C/DOS, MSVC): clrscr, getch, getche
`<math.h>`	maths on double: sqrt, pow, fabs, ceil, floor, sin, log (gcc: link with -lm)
`<string.h>`	strings and memory blocks: strlen, strcpy, strcat, strcmp, strstr, memcpy
`<stdlib.h>`	general utilities: malloc, calloc, free, atoi, rand, exit, abs, qsort
`<ctype.h>`	character tests and case: isalpha, isdigit, isspace, toupper, tolower
`<time.h>`	date and time: time, clock, difftime, CLOCKS_PER_SEC
`<limits.h>`	integer limits: INT_MAX, INT_MIN, CHAR_BIT, LONG_MAX
`<float.h>`	floating-point limits: FLT_MAX, DBL_MAX, FLT_EPSILON, FLT_DIG
`<stddef.h>` · `<stdbool.h>`	NULL, size_t, ptrdiff_t · bool, true, false (C99)

```
/* myheader.h */
#ifndef MYHEADER_H      /* include guard */
#define MYHEADER_H
int add(int a, int b); /* prototype */
#define MAX 100
#endif
/* main.c */
#include <stdio.h>      /* system dirs */
#include "myheader.h"   /* own dir 1st */
```

<…> vs "…" `<stdio.h>` is searched in the standard (system) include directories only — use it for library headers. `"my.h"` is searched in the **own folder first**, then the system directories — use it for your own headers. An **include guard** stops a second copy, so no "redefinition" errors; `#pragma once` does the same (non-standard, widely supported).

Memory trick "**Angle brackets = the library shelf; quotes = your own desk**, checked first, then the shelf."

Exam trap A header does **not** link its library: with `<math.h>` gcc still needs `-lm`, else "undefined reference". `conio.h`, `clrscr()`, `getch()` are not ISO C.

Worked example ✍ which header?

`INT_MAX` → *limits.h* · `toupper` → *ctype.h* · `malloc` → *stdlib.h* · `fabs` → *math.h* but `abs` → **stdlib.h**.

2 Preprocessor Directives and Macros

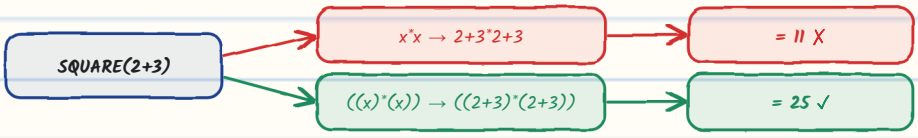
output questions ★

Definition The **preprocessor** runs **before the compiler** and handles every line that starts with # (a **directive**). It does pure text work — file inclusion, **macro** substitution, conditional compilation — after comments are stripped. A directive ends at the end of its line: **no semicolon** (continue a long one with \).

Directive	Purpose
#include	insert a file's text
#define	constant or macro: #define PI 3.1416
#undef	remove a macro definition
#ifdef · #ifndef	compile if defined · not defined
#if #elif #else #endif	compile by a constant integer test; defined(X) allowed
#error msg	stop compilation with a message
#pragma · #line	compiler-specific order (once, pack) · reset line number

```
#define DEBUG 1
#define VERSION 2
#ifdef DEBUG
  printf("Debug mode ON\n");
#endif
#if VERSION >= 2
  newApi();
#elif VERSION == 1
  oldApi();
#else
  #error "unknown VERSION"
#endif
#undef DEBUG /* gone from here */
```

Predefined macros: __FILE__ file name (string) · __LINE__ line number (int) · __DATE__ "Oct 14 2026" · __TIME__ "hh:mm:ss" · __STDC__ 1 on a conforming compiler · __STDC_VERSION__ 199901L (C99), 201112L (C11).



a macro is replaced as text; then normal precedence applies

Point	Macro	Function
Handled	preprocessing: text	run time: a call
Type check	none	yes (prototype)
Arguments	re-evaluated per use	evaluated once
Speed · size	no call cost · grows	call cost · one copy
Address	none	function pointer

and ## # stringizes: #define STR(x) #x, STR(Dhaka) → "Dhaka". ## pastes tokens: #define CAT(a,b) a##b, CAT(var,1) → var1. Safer than a function-like macro: an inline function (C99).

Memory trick "**Brackets inside, brackets outside**": wrap every parameter and the whole body — ((x)*(x)).

Exam trap #define PI 3.14; pastes the ; too. SQUARE(i++) increments i twice: undefined. #define SQ (x) with a space is object-like.

Worked example ✍ output

#define SQ(x) (x)*(x): 100/SQ(5) → 100/(5)*(5) = 20×5 = **100**, not 4. Fully bracketed: 4.

3 Pointers: &, *, Kinds and const

most tested ★

Definition A *pointer* is a variable that stores the **memory address** of another object: `type *name;`. `&` (**address-of**) gives an address; `*` (**indirection**, "value at") gives the object at an address. A pointer's own size is fixed by the machine — 4 bytes on 32-bit, 8 on 64-bit — whatever it points to; its type decides how many bytes `*` reads and how far `+1` moves.



*p = 10 · **q = 10 · *q = p = 1000 · &p = 2000

a pointer's box holds another box's address

```
int a = 10;
int *p;          /* declare      */
p = &a;          /* address of a */
printf("%d", *p); /* 10          */
*p = 20;         /* a is now 20  */
int **q = &p;    /* ptr to ptr   */
printf("%d", **q);/* 20          */
int *x, y;       /* x ptr, y int! */
```

Expression	Value
a · &a	20 · 1000
p · *p · &p	1000 · 20 · 2000
*&a (they cancel)	20
q · *q · **q	2000 · 1000 · 20
printf("%p", (void *)p)	prints 1000

Kind	What it is	Danger and fix
NULL	`int *p = NULL;` points nowhere	dereference → crash; test `p != NULL`
void (generic)	holds any object address; `malloc` returns one	no `*v` or `v+1` without a cast: `*(int *)v`
Wild	never initialised: garbage address	initialise to `NULL` or an address
Dangling	object gone: after `free(p)`, or a local's address returned	`p = NULL;` after `free`

Declaration	*p=5	p++
`const int *p` = `int const *p`	✗	✓
`int *const p = &a;`	✓	✗
`const int *const p`	✗	✗

Look-alikes `int *p[3]`: array of 3 pointers. `int (*p)[3]`: one pointer to an array of 3 ints. `int (*fp)(int)`: function pointer. Passing `&x` to a function: previous lesson.

Memory trick "**& asks where; * asks what's there.**" Read right to left: `int *const p` = "p is a const pointer to int".

Exam trap In `int *x, y;` only **x** is a pointer: `*` binds to the name. A NULL or wild dereference compiles, then fails at run time.

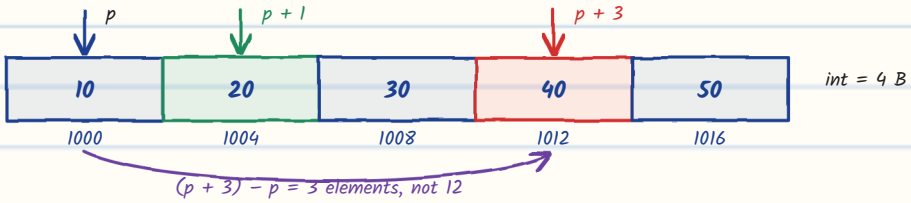
Worked example ✍ trace

`int a = 5, *p = &a, **q = &p; **q += *p;` → both name a, so a = 5 + 5 = 10.

4 Pointer Arithmetic

numeric MCQs ★

Definition Adding an integer n to a pointer moves it **n elements**, not n bytes: address(p + n) = address(p) + n × sizeof(*p). Subtracting two pointers into the same array gives the **number of elements** between them (type ptrdiff_t). Results are defined only inside one array, plus the position one past the end.



int a[5] = {10,20,30,40,50}; int *p = a; — each step jumps 4 bytes

Pointer type	sizeof(*p)	p+1 from 1000
char *	1	1001
short *	2	1002
int *, float *	4	1004
double *	8	1008
int ** (64-bit)	8	1008
12-byte struct *	12	1012

Operation	OK?	Result
p + n, p − n, p++, p--	✓	moves n elements
p2 − p1	✓	elements between
p1 < p2, p == NULL	✓	compare positions
p1 + p2	✗	compile error
p * 2, p / 2, p % 2	✗	compile error
void * arithmetic	✗	not ISO C (gcc: 1 B)

```
int a[] = {10, 20, 30, 40};
int *p = a;
int x = *p++;    /* x=10, p->a[1] */
int y = (*p)++; /* y=20, a[1]=21 */
int z = *++p;    /* z=30, p->a[2] */
printf("%d %d %d %d", x, y, z, a[1]);
                 /* 10 20 30 21   */
int *q = &a[3];
printf("%d", (int)(q - a)); /* 3 */
```

Expression	Changes	Value used
*p++ = *(p++)	**p**, after	old *p
*++p	**p**, first	new *p
(*p)++	**value**, after	old value
++*p	**value**, first	new value

Walking an array with a pointer for (p = a; p < a + n; p++) sum += *p; — comparing with a + n (one past the end) is legal, but reading *(a + n) is not. Pointers into **different** arrays must not be subtracted or ordered with <; == is always allowed. Print a pointer difference with %td.

Memory trick "**The type is the ruler.**" A pointer counts in elements; its type says how long one step is.

Exam trap *p++ moves the **pointer**, not the value: postfix ++ binds tighter than *. Two int pointers 12 bytes apart differ by **3**.

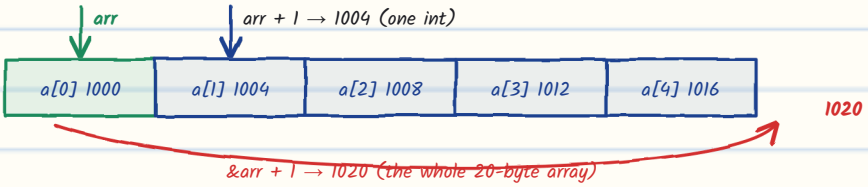
Worked example ✍ address

double *d holds 2000; d + 5 = 2000 + 5×8 = **2040**. As char *: 2005.

5 1D Arrays and the Array–Pointer Link

highest-yield ★

Definition An **array** is a fixed-size collection of elements of the **same type** in **contiguous** memory, indexed `0` … `n-1`; `int arr[5]` reserves `5 × 4 = 20` bytes. C does **no bounds checking**: `arr[5]` compiles and is undefined behaviour. In most expressions the name **decays** to a pointer to element 0, so **`arr[i] ≡ *(arr + i)`** `≡ *(i + arr) ≡ i[arr]`.



arr and &arr print the same number; their types and steps differ

Expression	Type	Value
`arr, &arr[0]`	`int *`	1000
`arr + 1`	`int *`	1004
`&arr`	`int (*)[5]`	1000 (same!)
`&arr + 1`	`int (*)[5]`	1020
`sizeof(arr)`	`size_t`	**20**
`sizeof arr / sizeof arr[0]`	`size_t`	5

Declaration	Contents
`int a[5]={1,2};`	1 2 0 0 0
`int a[]={1,2,3};`	size taken as 3
`int a[5]={0};`	all zero
`int a[5];` local · global	garbage · all zero
`int a[5]={[2]=7};`	0 0 7 0 0 (C99)
`int a[n];`	VLA: C99; optional in C11

```
int sum(int a[], int n) { /* int *a */
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];        /* *(a + i) */
    return s;  /* sizeof(a) = 8 here */
}
int arr[5] = {1, 2, 3, 4, 5};
int n = sizeof arr / sizeof arr[0];
sum(arr, n);   /* passes &arr[0] */
```

When the name does NOT decay ① under `sizeof` → whole array size; ② under `&` → pointer to the whole array; ③ a string literal initialising a `char` array. The name is **not a modifiable lvalue**: `arr++`, `arr = p` fail; `p++` is fine. A function gets only the address, so its changes to `a[i]` reach the caller.

Memory trick "**Array name = a fixed signpost to element 0; a pointer = a signpost you can carry.**"

Exam trap An array parameter is a pointer: `sizeof(a)` inside the function gives 8 (or 4), not the array size. Always pass `n`.

Worked example ✍

output

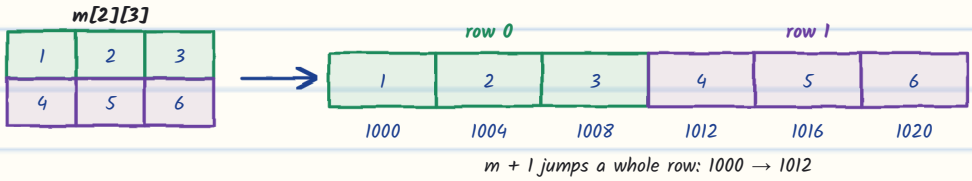
```
int a[5]={1,2,3,4,5};
int *p=(int *)(&a+1);
printf("%d",*(p-1)); →
```

5

6 2D Arrays and the Address Formula

written + numeric ★

Definition `int m[R][C]` is an array of R rows, each an array of C ints. Memory is one straight line, and C stores the elements in **row-major order**: all of row 0, then row 1, … Only the **first** (row) size may be left out — `int m[][3] = {…};`, `void f(int m[][3], int rows)` — because the **column size is required** to find where each row starts.



the grid is how we think; the line is how memory holds it

```
int m[2][3] = { {1, 2, 3},
                {4, 5, 6} };
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++)
        printf("%d ", m[i][j]);
    printf("\n");
}           /* 1 2 3
               4 5 6 */
int (*row)[3] = m; /* ptr to a row */
```

Expression	Means	Value
`m`	→ row 0	1000
`m + 1`	→ row 1 (+12 B)	1012
`*m + 1`	`&m[0][1]` (+4 B)	1004
`*(m + 1) + 2`	`&m[1][2]`	1020
`*(*(m + 1) + 2)`	`m[1][2]`	6
`sizeof m · m[0]`	all · one row	24 · 12

Address formula M rows, N columns, W bytes per element, base address B, lower bounds L_r, L_c (both 0 in C).

Row-major: $LOC(A[i][j]) = B + W \times [N \times (i - L_r) + (j - L_c)]$

Column-major: $LOC(A[i][j]) = B + W \times [M \times (j - L_c) + (i - L_r)]$

C, C++, Pascal and NumPy (default) are row-major; Fortran, MATLAB and R are column-major.

Worked example ✍ A[1..10][1..15], W = 2, B = 100; find A[4][8] M = 10, N = 15. **Row-major:** 100 + 2 × [15 × 3 + 7] = 100 + 2 × 52 = **204**. **Column-major:** 100 + 2 × [10 × 7 + 3] = 100 + 2 × 73 = **246**. In C, `int A[5][4]` at 1000: `&A[3][2]` = 1000 + 4 × (3×4 + 2) = 1056.

Worked example ✍ pointer to a row `int a[2][3] = {{1,2,3},{4,5,6}}; int (*r)[3] = a;` → `r[1][1]` = 5, `*(*(r + 1))` = **4** (first element of row 1), `*(*r + 2)` = 3. `r + 1` moves 12 bytes because `*r` is a whole row of 3 ints.

Memory trick "**Row-major reads like a book**": finish a line, then the next. Column-major reads a newspaper column.

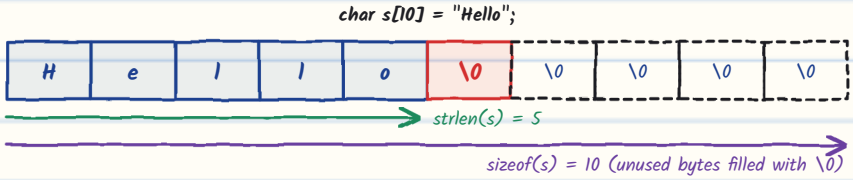
Exam trap `int m[][] = {…};` is an error. `m + 1` jumps a row, `*m + 1` one element. Check whether bounds start at 0 or 1.

Worked example ✍ size `int a[2][3][4]` holds 24 ints = **96 bytes**; `a[1]` is a 3×4 block of 48 bytes.

7 Strings and the Null Character

MCQ + output ★

Definition C has no string type: a **string** is a char array ending in the **null character '\0'** (ASCII 0). Every string function — strlen, printf("%s"), strcpy — walks until it meets '\0'. So the literal "Hello" occupies **6 bytes**: 5 letters + the terminator, added by the compiler.



strlen stops at the first '\0'; sizeof counts every byte the array owns

Declaration	Bytes	Note
char s1[] = {'H','i','\0'};	3	'\0' by hand
char s2[] = "Hello";	6	'\0' added
char s3[10] = "Hello";	10	rest '\0'
char s4[5] = "Hello";	5	**no '\0'**

Common misconception The terminator is '\0', value 0 — **not** ' ' (space, 32), '0' (digit, 48) or '\n' (10). NULL is a **pointer**; '\0' a **character**.

Point	char s[] = "Dhaka";	char *p = "Dhaka";
What it is	array: own copy, 6 bytes	pointer to a string literal
Change a letter	s[0] = 'd'; ✓	p[0] = 'd'; ✗ undefined (literal often read-only → crash)
Assign new text	s = "Khulna"; ✗ → strcpy	p = "Khulna"; ✓ re-points
sizeof	6	8 (pointer, 64-bit)

```
char s[] = "abc";
printf("%zu %zu", sizeof s, strlen(s)); /* 4 3 */
printf("%zu", sizeof("abc"));       /* 4 */
char *p = "Hello";
printf("%c %s", *(p + 1), p + 2);   /* e llo */
printf("%c", "Hello"[4]);           /* o */
char *names[] = {"Ali", "Rina"}; /* 2 ptrs */
```

'A' vs "A" 'A' is a character constant of type int in C: sizeof('A') = 4 (1 in C++). "A" is {'A','\0'}: sizeof("A") = 2. char *names[] = array of pointers; char g[2][5] = fixed 5-byte rows.

Memory trick "**Every string carries an invisible full stop**": bytes needed = letters + 1.

Exam trap s1 == s2 compares **addresses**, not text → strcmp. char t[] = {'H','i'}; has no '\0': %s runs on.

Worked example ✍ own strlen

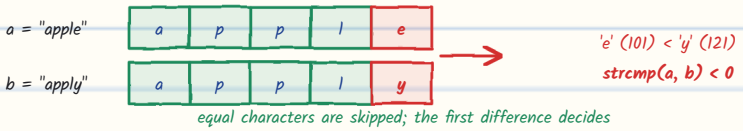
size_t len(const char *s){ const char *p = s; while (*p) p++; return p - s; } → difference = **count**.

8 string.h Functions and String Input

frequently tested ★

Definition <string.h> declares the functions for '\0'-terminated strings and raw memory blocks. None of them knows the destination's size: the caller must leave room, including the byte for '\0'. Comparison is **lexicographic by character code**.

Function	Does	Returns / note
strlen(s)	counts characters before '\0'	size_t; '\0' not counted
strcpy(d, s)	copies s with its '\0' into d	d; no size check
strncpy(d, s, n)	copies at most n chars	no '\0' if strlen(s) ≥ n
strcat · strncat(d, s, n)	appends s (≤ n chars) to d	d; strncat adds '\0'
strcmp(a, b) · strncmp	compares char by char	0 if equal, <0 or >0
strchr · strrchr · strstr	first c · last c · substring	pointer, or NULL
strtok · memcpy · memset	split · copy · fill raw bytes	strtok modifies s
strrev, strupr, itoa	reverse · upper case · int → text	not standard C



strcmp works like a dictionary, but by ASCII code

Input call	Stops at	Size check	Note
scanf("%s", s)	whitespace	no → "%49s"	no &; s is an address
scanf("%[^\n]", s)	newline	no	reads spaces too
gets(s)	newline	none	removed in C11
fgets(s, n, stdin)	newline (kept) or n−1	✓ safe	adds '\0'; NULL at EOF

```
char name[50];
fgets(name, sizeof name, stdin);
name[strcspn(name, "\n")] = '\0';
if (strcmp(name, "Rahim") == 0)
    puts("match");   /* puts adds \n */
char d[6];
strncpy(d, "Bangladesh", 5);
d[5] = '\0';         /* "Bangl" */
```

Why gets() went No size parameter: a long line writes past the array over neighbouring memory — a buffer overflow (the 1988 Morris worm exploited one in fingerd). Deprecated in C99 (TC3), removed in C11. More I/O: later lesson.

Memory trick "**strcmp = dictionary order by ASCII**": uppercase (65–90) sorts before lowercase (97–122).

Exam trap if (strcmp(a, b)) is true when the strings **differ**. Only the sign is guaranteed, not −1/+1.

Worked example ✍ mixed input

scanf("%d",&n); fgets(s,50,stdin); → fgets gets the leftover '\n': an **empty line**.

Night-Before-the-Exam Sheet

Fact sheet

<h>: system dirs · "h": own dir, then system

Directives: no ;, end at the line end

stringize · ## token paste

p + n = address + n × sizeof(*p)

arr[i] ≡ *(arr+i) ≡ i[arr]

Row-major: $B + W[N(i-L_r) + (j-L_c)]$

"Hello" = 6 bytes; '\0' = ASCII 0

Pipeline: preprocess → compile → assemble → link

Guard: #ifndef X / #define X / #endif

__FILE__ __LINE__ __DATE__ __TIME__

p2 − p1 = elements, not bytes

No decay: sizeof arr, &arr

Column-major: $B + W[M(j-L_c) + (i-L_r)]$

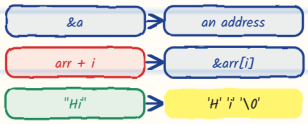
strcmp: 0 equal, <0, >0

→ Top 12 one-liners for MCQs

1. printf/scanf → stdio.h; malloc → stdlib.h
2. conio.h is not standard C
3. SQUARE(2+3) with x*x → 11
4. const int *p: value fixed; int *const p: pointer fixed
5. void * cannot be dereferenced without a cast
6. *p++ moves p; (*p)++ changes the value
7. Pointer size: 4 B (32-bit), 8 B (64-bit)
8. &arr + 1 skips the whole array
9. Array parameter = pointer; pass the length
10. Only the first dimension may be omitted
11. strlen("abc") = 3, sizeof("abc") = 4
12. gets removed in C11 → fgets

Top traps #define with ; · 100/SQ(5) = 100 · int *x, y; · dangling after free · sizeof of an array parameter · m[][] without columns · modifying char *p = "…" · == on strings · strncpy without '\0'.

All tricks together shelf vs desk · brackets inside, brackets outside · & asks where, * asks what's there · the type is the ruler · fixed vs carried signpost · row-major reads like a book · invisible full stop · strcmp = dictionary by ASCII.



three ideas, three lines

Worked example ✍ one snippet, five facts char s[] = "Dhaka"; char *p = s + 1; → sizeof s = 6, strlen(s) = 5, *p = 'h', p[2] = 'k', p − s = 1. And *s = 'd' is fine because s is an array, not a literal.

✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

1. With `#define SQUARE(x) x*x`, `SQUARE(3+1)` evaluates to:
(a) 16 (b) 8
(c) 7 (d) 10

2. Inside `void f(int a[])` on a 64-bit system, `sizeof(a)` is:
(a) 8 (b) 20
(c) 5 (d) 4

3. Which header declares `strlen()`?
(a) stdio.h (b) stdlib.h
(c) ctype.h (d) string.h

4. An `int *p` (4-byte int) holds 2000. `p + 3` is:
(a) 2003 (b) 2012
(c) 2006 (d) 2024

5. `const int *p;` means:
(a) `*p` is read-only (b) `p` is read-only
(c) both are fixed (d) neither is fixed

6. `int A[5][4]`, base 1000, 4-byte int, row-major. Address of `A[3][2]`:
(a) 1052 (b) 1048
(c) 1056 (d) 1060

7. Which function was removed from the C11 standard?
(a) fgets (b) gets
(c) puts (d) scanf

8. `char s[] = "Dhaka";` — `sizeof(s)` is:
(a) 5 (b) 4
(c) 8 (d) 6

9. `strcmp("abc", "abd")` returns:
(a) 0 (b) a positive value
(c) a negative value (d) 3

10. A header is kept from being included twice by:
(a) `#pragma pack` (b) `#ifndef` / `#define` / `#endif`
(c) `#error` (d) `#undef`

Answer key → 1-c · 2-a · 3-d · 4-b · 5-a · 6-c · 7-b · 8-d · 9-c · 10-b

Written question Explain the relationship between arrays and pointers in C. Show how `arr[i]` is evaluated, why `sizeof` gives different results for an array and a pointer, derive the row-major address of `A[i][j]`, and write a pointer-based function that returns the length of a string. 10 marks

1. **Decay (2)** — an array name in an expression becomes `&arr[0]`; `arr[i]` is defined as `*(arr + i)`, and `arr + i` adds `i × sizeof(int)` bytes. 2
2. **sizeof (2)** — no decay under `sizeof`/`&`: `int arr[5]` → 20; a pointer (or array parameter) → 8 on 64-bit; the name cannot be assigned or incremented. 2
3. **Address (3)** — rows stored one after another: skip i full rows of N elements, then j: $LOC = B + W[N(i - L_r) + (j - L_c)]$; e.g. `int A[5][4]` at 1000 → `A[3][2]` = 1056. 3
4. **Function (3)** — `size_t len(const char *s){ const char *p = s; while (*p) p++; return p - s; }`: walk to `'\0'`, pointer difference = length. 3

→ Before the exam ✓

- ✓ Can name the header of any common function
- ✓ Can expand a macro and spot the trap
- ✓ Can compute `p + n` and `p2 - p1`
- ✓ Can tell `arr`, `&arr`, `sizeof` apart
- ✓ Can apply both 2D address formulas
- ✓ Can count string bytes with `'\0'`